day/date

Computer Network
Assignment #2

Muhammad Hammad 24K-0544

Task 1

1. The satellite link loses, corrupts and delays packet and the network layer below only offers best-effort delivery. A lost casualty list or a corrupted rescue assignment could send a team to the wrong place, so the transport layer must guarantee that every packet arrives correctly, completely (ACK + transmission) and in order

2.

An example:- If you post a letter and refuse to write the next one until the reply arrives Send my one packet takes L/R = 8000/2000000 = 4ms but the sender then wait about 500 ms for the ACK. The link is idle about 99% of the time, which wastes a limited bandwidth link in an emergency

3.

The Sender transmit up to N packets before the first ACK returns, so the 500 ms of waiting is filled with useful transmission. Utilization grows roughly N times. The cost is wider sequence-number range and buffering the sender, and possibly at the receiver

Finally Bingo!

Scanned with CamScanner

day/date

4.

SR is more suitable for this scenario due to following

- Retransmission cost: with RTT = 500 ms and limited bandwidth, every wasted retransmission is expensive. GBN resends the whole window after one loss, while SR resends only the lost packet
- Errors delay: The channel also corrupts and delay packets, so out-of-order arrivals are common. SR buffers them and ACKs each one, while GBN discard good packets

Task 2.

Assumptions:-

1. The timeout for packet 2 fires after packet 4 is sent but before packet 5 is sent.
2. ACKs → 0, 1 arrive before timeout
3. Window is full-size N = 4, with no wrap around ⇒ numbers → 0-9

a)

Event	Sender	Packet	Receiver	ACK	Sender Window
1	Send 0	0			[0-3], base = 0 nextseqnum = 1
2	Send 1	1			nextseqnum = 2
3	Send 2	2 X (lost)			nextseqnum = 3
4	Send 3	3			nextseqnum = 4 (window full)
5		0 (arrives)	in order, deliver expectedseqnum = 1	ACK 0	
6		1 (arrives)	deliver expectedseqnum = 2	ACK 1	
7		3 (arrives)	expected = 2 ≠ 3 → discard	ACK 1 (re-Ack)	
8	Get ACK 0 → base = 1, send = 4	4			[1-4] nextseqnum = 5
9	Get ACK 1 → base = 2				[2-5]
	timer restarts for pkt 2				

Finally BingO!

Scanned with CamScanner

day/date

10		4 (arrives)	expected=2 → discard	ACK1 (dup)	
11	Dup ACKs*2 → ignored				base stays 2
12	Timeout (pkt 2) → go back, resend 2,3,4				[2-5]
13	Retransmit 2	2	deliver, expected=3	ACK2	[2-5]
14	Retransmit 3	3	deliver, expected=4	ACK3	[2-5]
15	Retransmit 4	4	deliver, expected=5	ACK4	[2-5]
16	Send 5 (new)	5	expected=6	ACK5	nextseqnum=6
17	ACK2 → base=3, send6	6	expected=7	ACK6	[3-6]
18	ACK3 → base=4 send 7	7	expected=8	ACK7	[4-7]
19	ACK4 → base=5 send8	8	expected=9	ACK8	[5-8]
20	ACK5 → base=6 send9	9	expected=10	ACK9	[6-9]
21	ACK6,7,8 → base=7,8,9				shrinks to [9]
22	ACK9 → base=10		all 10 delivered		done

b)

Event	Sender	Packet	Receiver	ACK	Sender window
1-4	Send 0,1,2,3	0, 1, 2✗, 3			S[0-3]; R[0-3]
	(2 lost)				
5	-	0 (arrives)	deliver, rcv_base=1	ACK0	R[1-4]
6		1 (arrives)	deliver, rcv_base=2	ACK1	R[2-5]
7		3 (arrives)	in window out of order → buffer	ACK3	rcv_base stays 2
8	ACK0 → base=1 send4	4			S[1-4]
9	ACK1 → base=2				S[2-5]
10		4 (arrives)	buffer	ACK4	R[2-5]
11	ACK3, ACK4 marked acked but base stays 2				S[2-5]
12	TIMEOUT (pkt 2's own timer)				
13		2 (arrives)	deliver 2, then flush buffered 3,4	ACK2	R[5-8]
			rcv_base=5		

Finally Bingo!

Scanned with CamScanner

day/date

14	ACK→2,3,4 all acked→ base=5; Send 5,6,7,8	5-8	each delivered in order	ACK 5-8	S[5-8]
15	ACK 5→base=6 Send 9	9	deliver rcv. base=10	ACK 9	S[6-9]
16	ACK 6-9→ base=10		all 10 delivered		done.

- GBN retransmit 3 packet (2,3,4) for 13 transmission in total
- SR retransmit 1 packet (2), for 11 transmissions in total

Task 5:

Transmission rate = $\frac{L}{R} = \frac{8000}{2\times10^6} = 0.004 = 4ms$

Stop-and-wait

$U = \frac{L/R}{RTT + L/R} = \frac{4}{500+4} = 0.0079 = 0.79\%$

Throughput = $U * R = 0.0079 * 2$
= 15.9 kbps

Pipelining N = 4

$U = N \cdot \left[\frac{L/R}{RTT + L/R}\right] = 4 \cdot \left[\frac{4}{504}\right] = 0.0317$
3.17%

Throughput = 63.5 kbps

Bing0!

Scanned with CamScanner

day/date

Stop-and-wait idles the link for a whole RTT after every packet. Pipelining uses that waiting time to send further packets, so the link is busy N times as long. N=4 is a 4x improvement.

2 k=3

a) $2^3=8$ Sequence number (0-7)

b) Maximum safe window: $2^{(k-1)} \Rightarrow 2^2=4$

c) IF N is large than 4, for eg N=5 the receiver cannot tell an old transmission from a new packet

Sequence number: 0 1 2 3 4 5 6 7

Sender's old window (N=5): [0 1 2 3 4]

Receivers new window (N=5): : [5 6 7 0 1]

· Overlap: 0 & 1 are amb. now

All 5 Acks are lost → Sender retransmits old pkt 0

Receiver sees Seq 0 inside its New window accepts it as new data

with N=4 old [0 1 2 3], new [4 5 6 7]

no overlap, no ambiguity.